

Circuite VLSI - proiect

Serial Command Router

Specificația completă a DUT-ului

Tip lucrare: Proiect individual
Limbaj: SystemVerilog (fără UVM)
Platforma: Vivado / EDAPlayground

Cuprins

1	Prezentare generală	2
1.1	Fișierele furnizate	2
2	Interfața externă	2
2.1	Porturile DUT-ului (router_top.v)	2
2.2	Reguli fundamentale ale protocolului	3
3	Protocolul serial — formatul pachetului	3
3.1	Structura pachetului	3
3.2	Câmpurile header-ului	4
3.3	Calculul parității	4
4	Formele de undă — comportament input/output	5
4.1	Forma de undă 1: Tranzacție WRITE	5
4.2	Forma de undă 2: Tranzacție READ	5
4.3	Forma de undă 3: Eroare de paritate	6
4.4	Forma de undă 4: Mid-packet drop (eroare de protocol)	6
4.5	Forma de undă 5: Silent Drop (WRITE pe slave lockat)	6
4.6	Forma de undă 6: Back-to-back (fără gap)	7
5	Comenzile suportate	7
5.1	Tabelul comenzilor	7
5.2	Detalii per comandă	7
5.2.1	WRITE (CMD=00)	7
5.2.2	READ (CMD=01)	7
5.2.3	INCREMENT (CMD=10)	8
5.2.4	RESET_SLAVE (CMD=11, ADDR=0/1/2)	8
5.2.5	LOCK și UNLOCK (CMD=11, ADDR=3)	8
6	Condițiile de eroare	9
A	Referință rapidă	9
A.1	Formule	9
A.2	Encodări rapide	9

1 Prezentare generală

Serial Command Router este un controler de comunicație care:

- Primește **pachete seriale byte-cu-byte** de la un singur Master extern
- Decodifică comenzile din header și le execută pe unul din **3 registre slave** de 8 biți
- Verifică **paritatea** pentru integritatea datelor
- Suportă operații: WRITE, READ, INCREMENT, RESET, LOCK, UNLOCK
- Implementează **Silent Drop**: comenzile pe un slave blocat sunt ignorate fără eroare
- Semnalizează **flow control** prin semnalul busy

1.1 Fișierele furnizate

Fișier	Rol în sistem
slave.v	Registru 8-bit cu unitate aritmetică, lock hardware și bus tri-state
router_fsm.v	FSM protocol serial: colectare bytes, verificare paritate, flow control
cmd_decoder.v	Decodificare comenzi, orchestrare tranzacții interne
bus.v	Bus Controller: handshake fizic cu slavi pe magistrala tri-state
router_top.v	Top-level: conectare module, definire magistrală tri-state

Tabela 1: Fișierele RTL furnizate

2 Interfața externă

2.1 Porturile DUT-ului (router_top.v)

Semnal	Dir.	Lăț.	Descriere
clk	IN	1b	Clock global. Toate elementele sincrone sampează pe frontul crescător
rst	IN	1b	Reset sincron, activ high. Readuce FSM-ul în IDLE și curăță registrele
packet_valid	IN	1b	Asertat în timpul header-ului și payload-ului . Trebuie să fie 0 la parity byte
data	IN	8b	Byte curent pe bus: header / payload / parity (byte-cu-byte)
ready	OUT	1b	Puls 1 ciclu: comanda executată cu succes
err	OUT	1b	Puls 1 ciclu: eroare de protocol sau paritate
busy	OUT	1b	DUT procesează un pachet. Masterul trebuie să aștepte
suspend_data_in	OUT	1b	Identic cu busy (alias). Când este 1, Masterul oprește transmisia
rdata	OUT	8b	Datele citite de la slavi în urma unei comenzi READ

Tabela 2: Porturile externe ale DUT-ului

Sincronizarea citirii (rdata): Semnalul `rdata` este expus direct la nivelul de top pentru a permite testarea de tip „black-box”. Valoarea prezentă pe acest port trebuie eșantionată de testbench **exact în același ciclu în care `ready` este 1**. În afara acestui puls, valoarea de pe `rdata` nu este garantată.

2.2 Reguli fundamentale ale protocolului

1. **Un byte per ciclu:** Masterul pune un byte pe `data` și asertează `packet_valid=1` timp de exact un ciclu de clock
2. **`packet_valid` cade înainte de parity:** La transmiterea parity byte-ului, `packet_valid` trebuie să fie **0**. Aceasta nu este o eroare, ci parte din protocol
3. **Mid-packet drop = eroare:** Dacă `packet_valid` cade la 0 în mijlocul payload-ului (înainte de finalizarea `LENGTH` bytes), DUT-ul returnează **err**
4. **Flow control:** Masterul verifică `suspend_data_in = 0` înainte de a trimite un pachet nou
5. **BFM pune date pe negedge:** Semnalele se asertează pe frontul descrescător pentru a fi stabile la samparea DUT-ului pe posedge
6. **Răspuns mutual exclusiv:** `ready` și `err` nu pot fi ambele 1 în același ciclu

3 Protocolul serial — formatul pachetului

3.1 Structura pachetului

Un pachet complet are **2 + LENGTH bytes**, transmiși consecutiv:

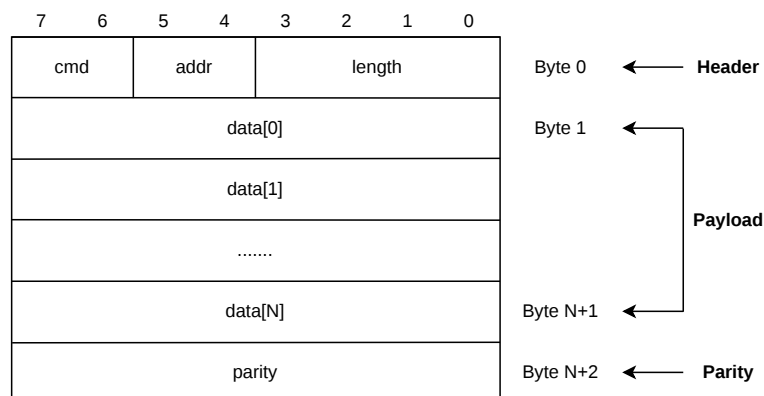


Figura 1: Structura pachetului

3.2 Câmpurile header-ului

Biți	Câmp	Lățime	Semnificație
[7:6]	CMD	2b	Tipul comenzii care va fi executată pe sclavul destinație. Valori posibile: 00 = WRITE, 01 = READ, 10 = INCREMENT, 11 = SPECIAL.
[5:4]	ADDR	2b	Adresa destinație. Valorile 0, 1, 2 selectează Sclavul 0, 1 sau 2 pentru operațiile normale. Valoarea 3 este rezervată exclusiv pentru comenzile de securitate (LOCK/UNLOCK) și este validă doar în combinație cu CMD=11.
[3:0]	LENGTH	4b	Numărul de bytes de payload transmiși după header (valori între 0 și 15 bytes).

Tabela 3: Câmpurile și encodarea header-ului

3.3 Calculul parității

$$\text{parity_byte} = \text{header} \oplus \text{payload}[0] \oplus \text{payload}[1] \oplus \dots \oplus \text{payload}[N - 1]$$

Verificare internă DUT (XOR cumulativ al tuturor bytes primiți, inclusiv parity):

$$\text{header} \oplus \text{payload}[0] \oplus \dots \oplus \text{payload}[N - 1] \oplus \text{parity} = 0x00 \Rightarrow \text{pachet valid}$$

4 Formele de undă — comportament input/output

Latența DUT-ului: De la ultimul byte (parity) până la `ready/err`, există o latență internă de **7–9 cicluri** (2 cicluri `cmd_decoder` + 3 cicluri `bus_controller` + 1–2 cicluri `slave` + 1 ciclu FSM RESPOND). Implementați timeouturi de **cel puțin 15–20 cicluri** în BFM.

4.1 Forma de undă 1: Tranzacție WRITE

Exemplu: WRITE 0xA5 pe slave 0, LENGTH=1

Header = {00, 00, 0001} = 0x01 Parity = 0x01 XOR 0xA5 = 0xA4

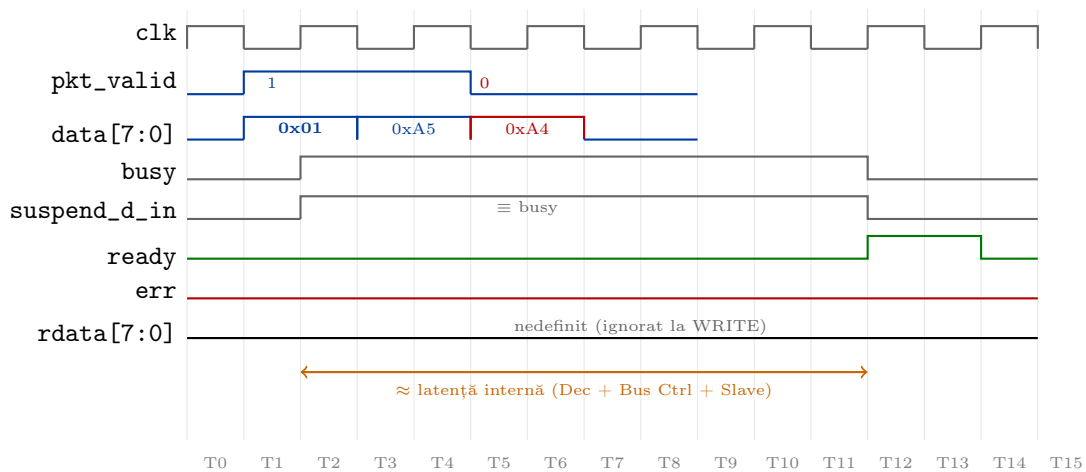


Figura 2: WRITE 0xA5 pe slave 0. `packet_valid` cade la 0 înainte de parity byte.

4.2 Forma de undă 2: Tranzacție READ

Exemplu: READ de pe slave 0 (după ce s-a scris 0xA5).

Header = {01, 00, 0001} = 0x41 Payload = orice byte (ignorat) Parity = 0x41 XOR P[0]

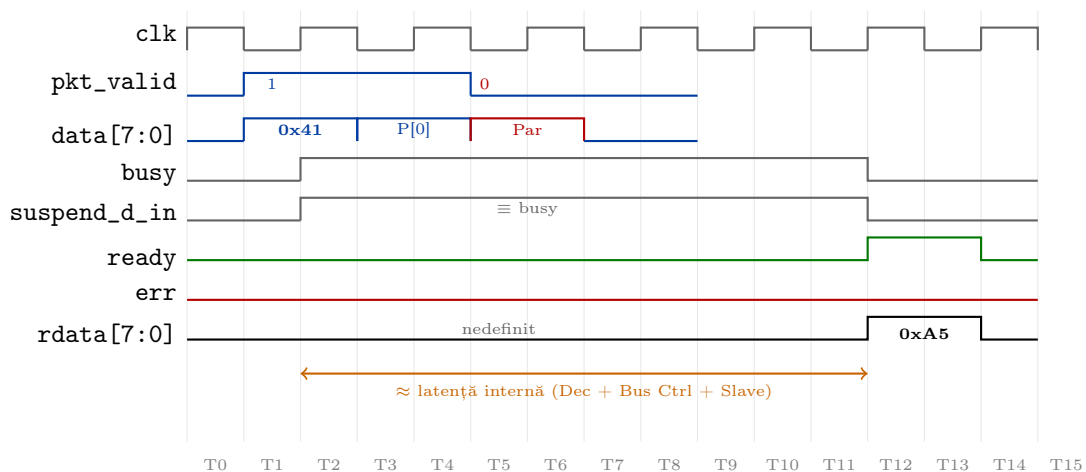


Figura 3: READ: `rdata[7:0] = 0xA5` este valid **simultan** cu pulsul `ready`

4.3 Forma de undă 3: Eroare de paritate

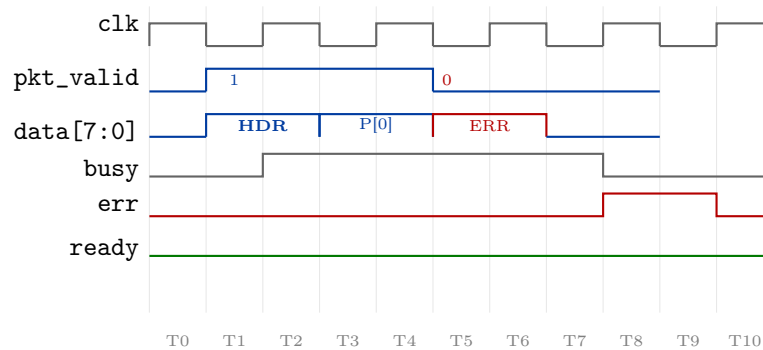


Figura 4: Eroare de paritate — `err` apare rapid, fără execuție pe slave

4.4 Forma de undă 4: Mid-packet drop (eroare de protocol)

Dacă `packet_valid` cade la 0 în mijlocul payload-ului, FSM-ul detectează eroarea.

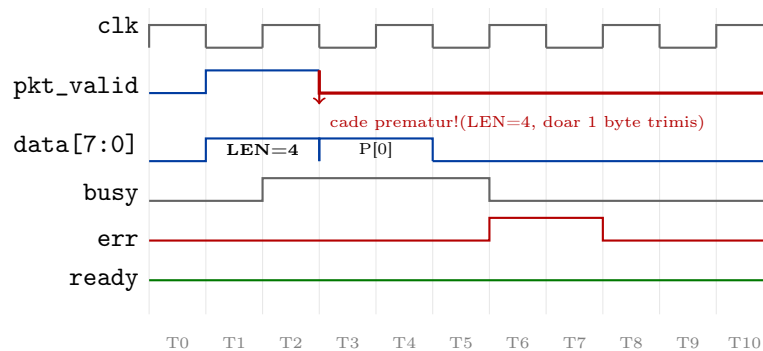


Figura 5: Mid-packet drop — `packet_valid` cade prematur în payload-ul așteptat

4.5 Forma de undă 5: Silent Drop (WRITE pe slave lockat)

Implicație critică pentru Scoreboard: La un WRITE pe un slave blocat, DUT-ul returnează `ready` ca și când scrierea ar fi reușit. Scoreboardul trebuie să știe starea de lock a fiecărui slave și să **nu actualizeze** modelul de referință în acest caz, chiar dacă vede `ready=1`.

4.6 Forma de undă 6: Back-to-back (fără gap)

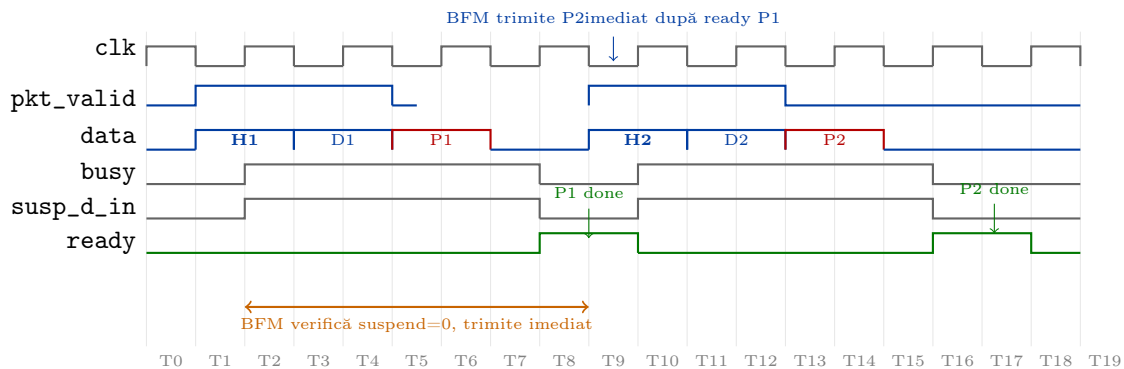


Figura 6: Back-to-back: Pachetul 2 trimis imediat după ready pachet 1, fără gap

5 Comenzile suportate

5.1 Tabelul comenzilor

Comandă	CMD	ADDR	Lock?	Efect
WRITE	00	0,1,2	Silent Drop	$\text{reg}[\text{ADDR}] \leftarrow \text{XOR}(\text{payload}[0], \dots, \text{payload}[\text{N}-1])$
READ	01	0,1,2	Permis	$\text{rdata} \leftarrow \text{reg}[\text{ADDR}]$ (simultan cu ready)
INCREMENT	10	0,1,2	Silent Drop	$\text{reg}[\text{ADDR}] \leftarrow (\text{reg}[\text{ADDR}] + \text{payload}[\text{N}-1]) \bmod 256$
RESET_SLAVE	11	0,1,2	Permis	$\text{reg}[\text{ADDR}] \leftarrow 0x00$, $\text{lock} \leftarrow 0$
LOCK	11	3	N/A	$\text{lock}[\text{payload}[1:0]] \leftarrow 1$
UNLOCK	11	3	N/A	$\text{lock}[\text{payload}[1:0]] \leftarrow 0$

Tabela 4: Comenzile DUT-ului. “Silent Drop” = returnează ready dar registrul nu se modifică.

5.2 Detalii per comandă

5.2.1 WRITE (CMD=00)

- **Efect:** $\text{reg}[\text{ADDR}] \leftarrow \text{XOR}$ al tuturor bytes din payload
- **Multi-byte:** $0x12, 0x34, 0x56 \rightarrow 0x12 \text{ XOR } 0x34 \text{ XOR } 0x56 = 0x70$
- **Single-byte:** $0xA5 \rightarrow \text{reg} = 0xA5$
- **Dacă slave e lockat:** **Silent Drop** — returnează ready, registrul rămâne nemodificat
- **rdata:** irelevant (nu se citește)

5.2.2 READ (CMD=01)

- **Efect:** niciun efect asupra registrului

- **Payload:** minim 1 byte (ignorat de DUT)
- **Dacă slave e lockat:** **permis** — returnează **ready** și valoarea curentă
- **rdata:** valid în ciclul **ready**

5.2.3 INCREMENT (CMD=10)

- **Efect:** $\text{reg}[\text{ADDR}] \leftarrow (\text{reg}[\text{ADDR}] + \text{payload}[\text{N}-1]) \bmod 256$
- **Atenție:** Folosește **ultimul byte** din payload, nu XOR-ul tuturor
- **Multi-byte:** $\{0xAA, 0x10\} \rightarrow$ incrementare cu $0x10$ (ultimul)
- **Wraparound garantat:** $0xFF + 1 = 0x00$
- **Dacă slave e lockat:** **Silent Drop**

5.2.4 RESET_SLAVE (CMD=11, ADDR=0/1/2)

- **Efect:** $\text{reg}[\text{ADDR}] \leftarrow 0x00$ și $\text{lock}[\text{ADDR}] \leftarrow 0$
- **Permis chiar și pe slave blocat** (singurul mod de a ieși din lock)
- **Payload:** minim 1 byte (ignorat)
- **Răspuns:** **ready**

5.2.5 LOCK și UNLOCK (CMD=11, ADDR=3)

Adresa specială 3 semnalează o comandă de securitate. Slave-ul țintă și operația sunt encodeate în payload:

payload[7:6]	payload[5:2]	payload[1:0]	Operație
00	ignorat	ss	LOCK slave ss (0, 1 sau 2)
01	ignorat	ss	UNLOCK slave ss
altceva	—	—	Fallback RESET (comportament nedefinit în spec)

Tabela 5: Encodarea LOCK/UNLOCK în payload[0]

Exemple concrete:

- $\text{payload}[0] = 0x02 = 0b00000010$: LOCK slave 2
- $\text{payload}[0] = 0x42 = 0b01000010$: UNLOCK slave 2
- $\text{payload}[0] = 0x01 = 0b00000001$: LOCK slave 1
- $\text{payload}[0] = 0x41 = 0b01000001$: UNLOCK slave 1

6 Condițiile de eroare

Eroare	Condiție	Tip
Paritate incorectă	$\text{XOR}(\text{header}, \text{payload}, \text{parity}) \neq 0x00$	Protocol
Adresă ilegală	$\text{CMD} \in \{00, 01, 10\}$ cu $\text{ADDR}=3$	Semantic
Mid-packet drop	<code>packet_valid</code> coboară în mijlocul payload-ului	Protocol

Tabela 6: Condițiile care generează `err`

Comportament comun la eroare:

- `err` pulsează exact **1 ciclu**
- `busy` revine la 0 în același ciclu cu `err`
- FSM revine în **IDLE** — gata să accepte un nou pachet
- Starea slavilor rămâne **nemodificată**
- `ready` și `err` sunt **mutual exclusive**

Ce **NU** generează `err`: WRITE sau INCREMENT pe un slave blocat returnează `ready` (Silent Drop), **nu** `err`. Scoreboardul trebuie să știe starea de lock a fiecărui slave pentru a verifica corect dacă registrul s-a modificat.

A Referință rapidă

A.1 Formule

$$\text{header} = \{\text{CMD}[1:0], \text{ADDR}[1:0], \text{LENGTH}[3:0]\} \quad \text{parity} = \text{header} \oplus \bigoplus_{i=0}^{N-1} \text{payload}[i]$$

$$\text{WRITE: reg} \leftarrow \bigoplus_{i=0}^{N-1} \text{payload}[i] \quad \text{INC: reg} \leftarrow (\text{reg} + \text{payload}[N-1]) \bmod 256$$

A.2 Encodări rapide

CMD	Comandă	ADDR valid
00	WRITE	0, 1, 2
01	READ	0, 1, 2
10	INCREMENT	0, 1, 2
11	SPECIAL	0,1,2=RESET_SLAVE; 3=LOCK/UNLOCK
	p[7:6]	p[1:0] CMD=11, ADDR=3
	00	ss LOCK slave ss
	01	ss UNLOCK slave ss
	altceva	— Comportament fallback